

Beyond Brownian Motion

Brownian motion is the backbone of this site, but real randomness also *jumps*, and real signals are usually seen only through noise. This chapter covers two classic extensions: processes with jumps, and filtering a hidden signal out of noisy measurements.

8.1 Jumps

Brownian paths never jump: over a short time they can only move a little (section 1.3). Market crashes, insurance claims and spikes in neural activity do jump. The simplest fix adds jumps at random moments: a *Poisson process* decides *when*, and each jump has a random size.

Predict first: Keep the total variance fixed, but put most of it into rare, big jumps. Compared with a bell curve, the distribution of end values will...

Look the same

Have a taller peak and much fatter tails

Become flat and wide

Answer: "Have a taller peak and much fatter tails". Rare, big jumps pile most paths near the middle (little happened) while a few land far out (a big jump hit), giving a taller peak and fatter tails than a bell curve with the same variance.

The total variance at $t = 1$ is fixed at 1. The sliders set how often jumps happen and what share of the variance they carry. The right panel uses a *log scale*, so rare extreme values stay visible next to the bell curve.

$$X_t = \sigma W_t + \sum_{k=1}^{N_t} J_k, \quad \text{Var}(X_t) = (\sigma^2 + \lambda \mathbb{E}[J^2]) t, \quad [X]_t = \sigma^2 t + \sum_{k=1}^{N_t} J_k^2$$

N_t A *Poisson process*: the number of jumps that have happened by time t . Jumps arrive at random moments, on average λ per unit of time, independently of each other.

λ The jump rate ("lambda"): how many jumps per unit of time on average.

J_k The size of jump number k : here a bell-curve number with mean 0, so jumps go up or down.

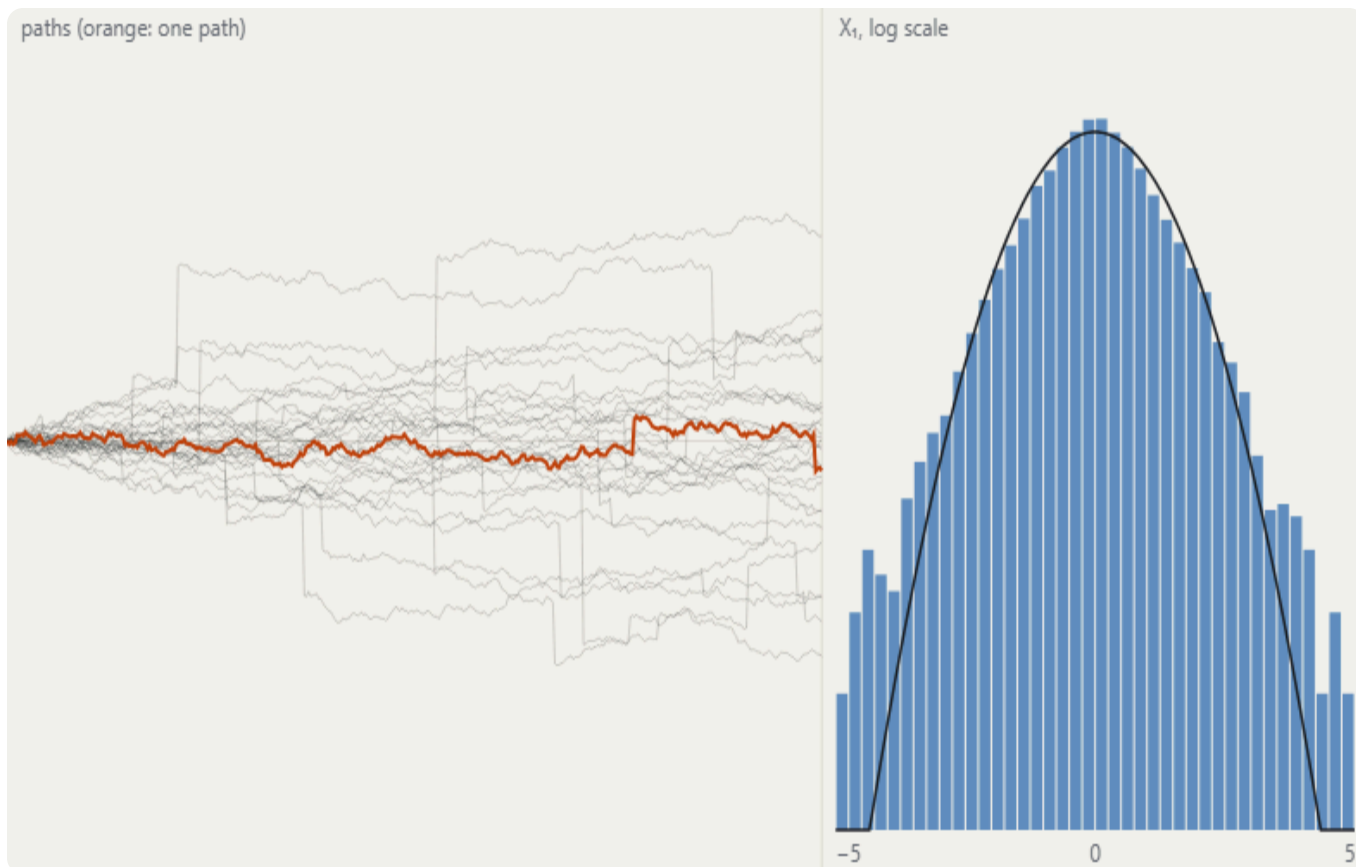
σW_t The ordinary Brownian part, scaled by σ .

$\text{Var}(X_t)$ The variance: the Brownian part contributes $\sigma^2 t$ and the jumps contribute $\lambda \mathbb{E}[J^2] t$.

$[X]_t$ The quadratic variation (section 2.1). Now it is *random*: $\sigma^2 t$ plus the sum of the squared jumps.

log scale Each step up the vertical axis multiplies by 10, so a bar ten times shorter is still easy to see.

In plain words: a few big jumps and a steady Brownian wobble can have the same variance but look very different. With rare big jumps, most paths are quiet, but the ones that jump land far away. That is a *fat tail*: extreme outcomes are many times more common than a bell curve predicts. Push the jump rate up while keeping the variance fixed, and the jumps get smaller and more frequent until the process looks Brownian again (the central limit theorem at work). One more change from Brownian motion: the quadratic variation is no longer simply t , because each jump adds its square.



Kurtosis of X_1 : 4.25 (bell curve: 3; theory here: 3.96). Share of $|X_1| > 3$: 0.83% (bell curve: 0.27%). Orange path: 5 jumps; its sum of squared steps $0.454 \approx \sigma^2 t + \sum J^2 = 0.425$.

Jumps create fat tails: the same variance, far more extreme outcomes. *Kurtosis* (the average fourth power, 3 for a bell curve) measures it, and here it equals $3 + 3s^2/\lambda$, where s is the share of variance from jumps. Rare big jumps give high kurtosis, while frequent small ones bring it back toward 3. That is why option markets price crash risk above what Black–Scholes predicts (section 6.1).

Going deeper: the precise statement

Lévy processes. Processes with independent, identically distributed increments are called Lévy processes. The Lévy–Itô decomposition splits every one of them into a drift, a Brownian part and a (compensated) sum of jumps, so this section's model is a typical member of the family.

Itô's formula with jumps. For $dX = \sigma dW + dJ$:

$$f(X_t) = f(X_0) + \int_0^t f'(X_s) \sigma dW_s + \frac{1}{2} \int_0^t f''(X_s) \sigma^2 ds + \sum_{s \leq t} [f(X_s) - f(X_{s-})].$$

The continuous part follows Chapter 4. Each jump adds its full change in f , not a Taylor approximation.

In finance. Merton's jump-diffusion model (1976) adds normally distributed log-price jumps to Black–Scholes. The hedging argument of section 6.1 then breaks down, because a jump can't be hedged with the stock alone.

References: R. Merton, "Option pricing when underlying stock returns are discontinuous", *J. Financial Economics* 3 (1976), 125–144. R. Cont & P. Tankov, *Financial Modelling with Jump Processes* (Chapman & Hall/CRC, 2004). Shreve, *Stochastic Calculus for Finance II*, ch. 11.

Show the code (Python + NumPy)

```
import numpy as np
rng = np.random.default_rng(23)
lam, share, paths = 2.0, 0.8, 200_000          # 2 jumps per unit time carry 80% of
the variance
sigma, delta = np.sqrt(1 - share), np.sqrt(share / lam)
n = rng.poisson(lam, paths)                    # number of jumps by t = 1, per path
X1 = sigma * rng.normal(size=paths) + delta * np.sqrt(n) * rng.normal(size=paths)  # sum
of n N(0, delta^2) jumps
print(f"variance {X1.var():.3f} (1)    kurtosis {np.mean(X1 ** 4) / X1.var() ** 2:.2f}
(theory {3 + 3 * share ** 2 / lam:.2f}; bell curve 3)")
print(f"share of |X1| > 3: {np.mean(np.abs(X1) > 3):.4f}    (bell curve 0.0027)")
```

Output when we ran it:

```
variance 1.006 (1)    kurtosis 3.97 (theory 3.96; bell curve 3)
share of |X1| > 3: 0.0079    (bell curve 0.0027)
```

8.2 The Kalman–Bucy Filter

Often the process you care about is *hidden*: a satellite's true position, a patient's true blood sugar, the "fair" value behind a noisy price. You only get noisy measurements of it. *Filtering* means estimating the hidden process, as it moves, from everything measured so far. For linear models like this one, the best possible estimate is itself the solution of an SDE: the Kalman–Bucy filter.

Predict first: Make the measurement noise r much larger. The filter's estimate will...

Follow the noisy readings more closely

Become smoother and lag behind, with a wider uncertainty band

Stay exactly the same

Answer: "Become smoother and lag behind, with a wider uncertainty band". Noisier measurements deserve less trust, so the filter leans more on its own prediction: the estimate gets smoother, reacts later, and its uncertainty band widens.

The hidden signal (thick grey) is an Ornstein–Uhlenbeck process (section 5.2). The blue dots are noisy readings of it. The orange line is the filter's running estimate, and the shaded band is its own measure of uncertainty. Drag the measurement noise.

$$dm_t = -\theta m_t dt + \frac{P_t}{r^2} (dY_t - m_t dt), \quad \frac{dP_t}{dt} = -2\theta P_t + \sigma^2 - \frac{P_t^2}{r^2}$$

X_t The hidden signal, following $dX = -\theta X dt + \sigma dW$ with $\theta = \sigma = 1$.

dY_t What is measured in a tiny time step: $X_t dt + r dV_t$, the signal plus independent noise dV .

r How noisy the measurements are (the slider).

m_t The filter's estimate: the best guess of X_t given every measurement up to time t .

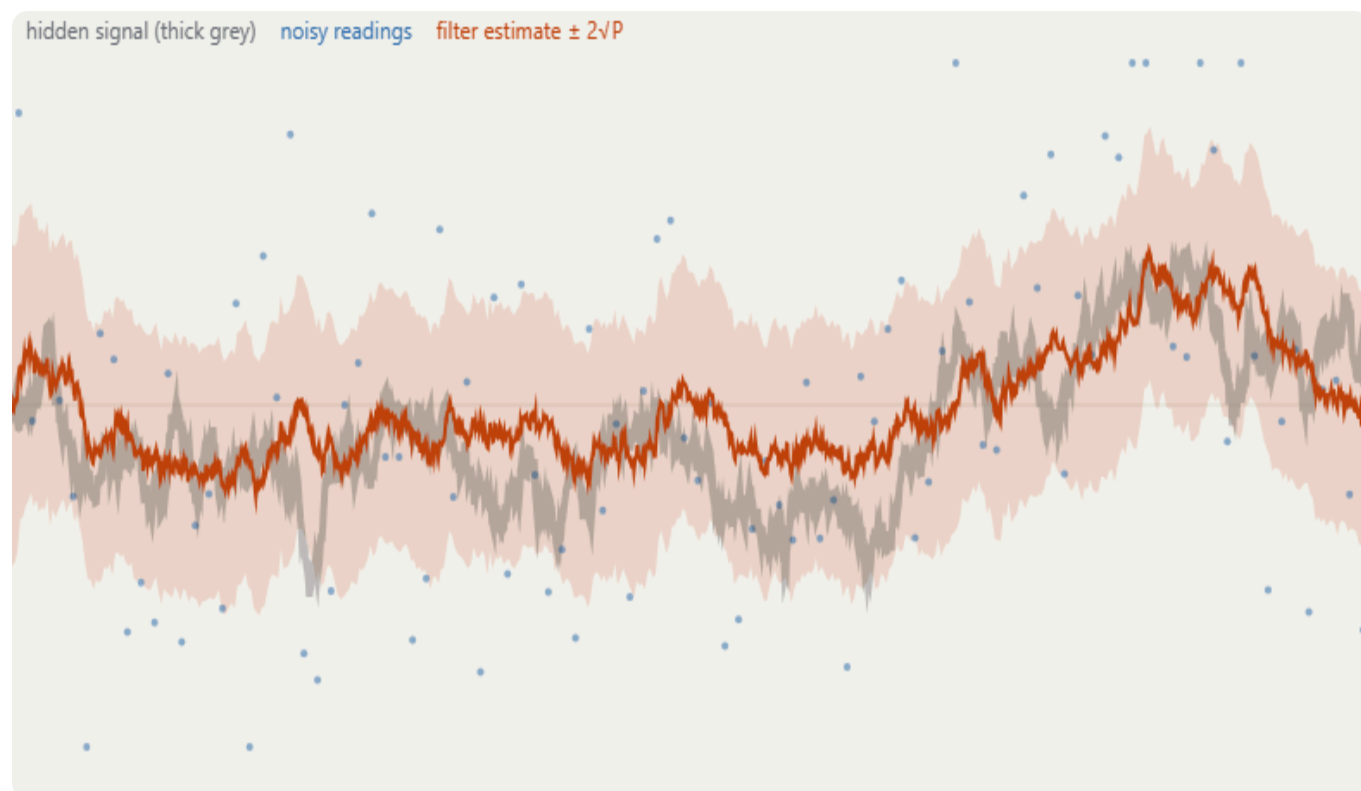
P_t The variance of the estimate's error: how unsure the filter is. The band is $m_t \pm 2\sqrt{P_t}$.

$dY_t - m_t dt$ The *innovation*: the part of the new measurement that the estimate did not predict, the "surprise".

P_t/r^2 The *Kalman gain*: how much of the surprise to believe. It is large when the filter is unsure (P big) or the data are clean (r small).

In plain words: between measurements, the estimate moves the way the signal is expected to move (the $-\theta m dt$ term). When a measurement arrives, the estimate is nudged toward it, by an amount that weighs the filter's own uncertainty against the measurement's noise. The second equation tracks that uncertainty. It grows because the hidden signal keeps getting random kicks (σ^2), and shrinks because measurements keep arriving ($-P^2/r^2$), until the two balance. The

estimate is then more accurate than any single reading, because it combines all of them with the model.



Typical error: filter 0.450, raw readings 1.407, guessing 0 with no data 0.683. The truth stayed inside the band 99.1% of the time (theory: about 95% on average; a single path varies). Long-run band half-width $2\sqrt{P_\infty} = 1.112$.

The filter beats the raw readings and knows its own uncertainty: the truth stays inside its band about 95% of the time, as a correct ± 2 -standard-deviation band should. Kalman filtering runs inside GPS receivers, spacecraft navigation (it flew on Apollo), weather forecasting and economic nowcasting.

Going deeper: the precise statement

Optimality. For linear signal and observation equations with Gaussian noise, the conditional distribution of X_t given the observations up to t is exactly normal, with mean m_t and variance P_t . No other estimator built from the same data has a smaller mean squared error.

The innovation is Brownian motion. The scaled innovations $\frac{1}{r}(dY_t - m_t dt)$ form a Brownian motion. The filter extracts everything predictable from the data, and what remains is pure, independent noise.

Beyond linear models. For nonlinear models the conditional distribution is no longer normal. It solves a stochastic PDE (the Kushner–Stratonovich or Zakai equation), and in practice it is approximated by

the extended Kalman filter or by particle filters.

References: R. Kalman, "A new approach to linear filtering and prediction problems", *J. Basic Engineering* 82 (1960), 35–45. R. Kalman & R. Bucy, "New results in linear filtering and prediction theory", *J. Basic Engineering* 83 (1961), 95–108. Øksendal, *Stochastic Differential Equations*, ch. 6.

Show the code (Python + NumPy)

```
import numpy as np
rng = np.random.default_rng(24)
theta, sigma, r, T, steps = 1.0, 1.0, 0.5, 10.0, 2000
dt = T / steps
x = m = 0.0
P = sigma ** 2 / (2 * theta) # no information yet
err = []
for _ in range(steps):
    dY = x * dt + r * np.sqrt(dt) * rng.normal() # noisy observation of the hidden
    signal
    m += -theta * m * dt + (P / r ** 2) * (dY - m * dt) # Kalman-Bucy update of the
    estimate
    P += (-2 * theta * P + sigma ** 2 - P ** 2 / r ** 2) * dt # Riccati equation for its
    variance
    x += -theta * x * dt + sigma * np.sqrt(dt) * rng.normal() # the hidden signal moves
    on
    err.append(m - x)
p_inf = r ** 2 * (np.sqrt(theta ** 2 + sigma ** 2 / r ** 2) - theta)
print(f"filter error spread {np.std(err):.3f}    theory sqrt(P_inf) = {np.sqrt(p_inf):.3f}
no-data guess {np.sqrt(0.5):.3f}")
```

Output when we ran it:

```
filter error spread 0.506    theory sqrt(P_inf) = 0.556    no-data guess 0.707
```

Chapter recap

Quick questions on this chapter's big ideas. Each answer is explained as soon as you choose.

1. A Poisson process has rate $\lambda = 3$ jumps per year. On average, how many jumps happen in two years?

3

6

1.5

Answer: "6". Jumps arrive at rate λ per unit of time, so on average $\lambda \times t = 3 \times 2 = 6$.

2. Compared with Brownian motion, the quadratic variation of a process with jumps is...

Always exactly t

Random, because every jump adds its square

Zero

Answer: "Random, because every jump adds its square". $[X]_t = \sigma^2 t + (\text{sum of squared jumps})$, and the jumps are random.

3. In the Kalman–Bucy filter, the gain P/r^2 is large when...

The measurements are noisy and the filter is already sure

The filter is unsure or the measurements are clean

Never: it is a fixed constant

Answer: "The filter is unsure or the measurements are clean". A large P (unsure) or a small r (clean data) means new measurements deserve more trust.